

Part 9: Loops

1. **Which instructions are used for loops in 8086?** The main ones are `LOOP`, `LOOPE` (Loop if Equal), and `LOOPNE` (Loop if Not Equal).
2. **How does `LOOP` instruction work?** It automatically decrements the `CX` register by 1. If `CX` is not zero, it jumps back to the specified label; otherwise, it moves to the next line.
3. **How do you implement a counting loop?** Move the count into `CX`, write a label at the start of the code block, and place the `LOOP` instruction pointing to that label at the end.
4. **How do you implement an infinite loop?** Create a label and use an unconditional `JMP` to jump back to it repeatedly. Example: `Start: ... JMP Start`.
5. **How do you implement a loop with `JCXZ`?** `JCXZ` (Jump if CX Zero) is used at the very start to skip the loop entirely if `CX` is already 0, preventing errors.
6. **Which register controls the number of iterations?** The `CX` (Count) register is the dedicated loop counter.
7. **How do you nest loops in 8086?** Push the outer loop's `CX` value onto the stack (`PUSH CX`) before starting the inner loop, and pop it back (`POP CX`) after the inner loop finishes.
8. **How can loops be used to traverse arrays?** Set `CX` to the array length, and use `SI` or `DI` to index elements. Inside the loop, increment the index register to move to the next item.
9. **How do you decrement a counter in a loop?** The `LOOP` instruction decrements `CX` automatically. If using a custom counter, you must manually use `DEC register`.
10. **How do you terminate a loop on a condition?** Use a `CMP` instruction inside the loop followed by a conditional jump (like `JE exit_label`) to break out early.
11. **How do you use `CMP` inside a loop?** Place `CMP` inside the loop body to check a value. If the condition is met, use a jump instruction to exit or skip code.
12. **How do you use `JMP` to create a loop?** Write a label, execute your code, modify a counter/check condition, and use `JMP` to go back to the label if the loop should continue.
13. **How do loops affect the `CX` register?** Every time the `LOOP` instruction executes, the value in `CX` decreases by exactly 1.
14. **How do you implement a sum of numbers using a loop?** Initialize a sum register (like `AX`) to 0. Loop `CX` times, adding the current number to `AX` on each iteration.
15. **How do you implement a factorial using a loop?** Initialize `AX` to 1 and `CX` to the number (e.g., 5). Loop, multiplying `AX` by `CX`, until `CX` becomes 1.